Architecture Justification

Problem Statement: Traffic Management System (TMS)

Architecture Selection

We chose an event-driven **Microservices Architecture** for the Traffic Management System. The system is split into independently deployable services: User & Auth, Traffic Monitoring, Route Planning, Parking, Incident, Signal Control, Alert & Notification, Analytics & Report, Audit Log and Traffic Data Ingestion. They are fronted by an API Gateway and connected through an Event Bus. Each service maps to a coherent group of SRS requirements (for example, the Incident Service covers FR-08 and FR-09).

Style Assessment for TMS

Style	Assessment for TMS	Verdict
Layered	Simple and familiar, but everything ships as one deployable. Adding roads, sensors or parking areas (NFR-06) or changing one feature (NFR-07) means redeploying the whole system, and a fault in analytics or data intake sits in the same process as signal control and emergency routing.	Not chosen
Client-Server	One central server is a bottleneck and single point of failure; a heavy report (FR-18) could slow down emergency routing (FR-15). Future sensor and IoT integration (NFR-11) would have to be bolted into that one server.	Not chosen
Microservices	Modular services, fault isolation for critical functions, and an event bus that suits real-time alerts and future sensor integration match the SRS quality requirements.	**Chosen**

Two Reasons for the Choice

- **Modularity and extensibility (NFR-06, NFR-07, NFR-11).** Each feature group is its own service behind a stable interface, so a feature such as Parking (FR-16) can be modified without touching the others. New roads, users and parking areas are added as data, and new data sources only need to publish to the Event Bus through the Ingestion Service. The simulated traffic feed can later be replaced by real sensors or IoT devices without changing any consumer service.
- **Fault isolation for critical functions (NFR-03, NFR-05, NFR-09).** Heavy or failure-prone work such as analytics (FR-17), report generation (FR-18) and data intake is separated from safety-critical work such as signal control (FR-07) and emergency priority and routing (FR-14, FR-15). A crash or slowdown in Analytics cannot take down Signal Control. Each service owns its own data store (FR-19), so a failed write in one service cannot corrupt the records of another.

Security Advantage

All external traffic passes through the API Gateway, which authenticates users and enforces role-based access (FR-02, FR-03, NFR-01). Administrator functions (FR-06, FR-07, FR-09, FR-18) and emergency functions (FR-14) are therefore unreachable for unauthorized roles, and internal services are never exposed directly. Only the User & Auth Service handles passwords and issues session tokens; other services see only a validated role. Personal and location data (NFR-10) stays inside the services that need it. A separate Audit Log Service receives signal changes, incident updates and alert creation as events and stores them with action and time (FR-20, NFR-12), so a compromised feature service cannot quietly rewrite its own trail.

Performance Benefit

Traffic events are handled asynchronously. The Traffic Monitoring and Incident services publish events to the Event Bus, and the Alert & Notification Service consumes them to raise alerts (FR-10) and notify users in selected or nearby areas (FR-11) without blocking any user request or polling. Read-heavy services such as Traffic Monitoring (FR-04, FR-05) and Route Planning (FR-12, FR-13) can be scaled out independently during peak hours while light services such as Parking stay small. Route Planning reads current congestion through its ITrafficData interface, so recommendations use the latest data (FR-13). Together this supports responsive normal requests (NFR-02) and availability during operating periods (NFR-03); the SRS gives no numeric response-time target, so one would be fixed during testing.

Trade-off Acknowledged

Microservices add operational complexity, network latency and eventual (rather than immediate) consistency between services. For TMS this is acceptable because the workload is naturally event-driven (traffic updates, incidents, alerts) and the benefits in modularity, isolation and future integration outweigh the extra deployment effort. For the prototype, the services can run as lightweight containers on a single machine.